
Network Optimization Models Study Set

1/25

Question 1. *A carpenter, plumber, and engineer are available to perform certain tasks. Each person can perform only one task in the allotted time. There are four tasks available, three of which must be done. Which person should be assigned to which job?*

- *Which job will go unfinished?*
- *Now suppose that each person can perform up to two tasks, and all tasks must be done. What should they do?*

Hint: Create a dummy person.

	SOLDERING	FRAMING	DRAFTING	WIRING
Carpenter	4	6	4	4
Plumber	3	4	2	3
Engineer	7	5	6	5

Table 1: The inefficiency matrix for person i assigned to task j

Network Optimization Models Study Set

2/25

Correct Answer 1. Assign tasks to individuals to minimize total inefficiency, subject to each individual only being able to perform one task in the first scenario, and up to two tasks in the second scenario.

LP Formulation for the First Scenario

Objective:

$$\begin{aligned} \text{Minimize } Z = & 4X_{CS} + 6X_{CF} + 4X_{CD} + 4X_{CW} + \\ & 3X_{PS} + 4X_{PF} + 2X_{PD} + 3X_{PW} + \\ & 7X_{ES} + 5X_{EF} + 6X_{ED} + 5X_{EW} \end{aligned}$$

Subject to:

$$\begin{aligned} X_{CS} + X_{CF} + X_{CD} + X_{CW} &= 1 \\ X_{PS} + X_{PF} + X_{PD} + X_{PW} &= 1 \\ X_{ES} + X_{EF} + X_{ED} + X_{EW} &= 1 \\ X_{CS} + X_{PS} + X_{ES} &\leq 1 \\ X_{CF} + X_{PF} + X_{EF} &\leq 1 \\ X_{CD} + X_{PD} + X_{ED} &\leq 1 \\ X_{CW} + X_{PW} + X_{EW} &\leq 1 \\ X_{ij} &\geq 0 \quad \text{and integer (for all } i, j) \end{aligned}$$

Where X_{ij} is 1 if person i is assigned to task j , and 0 otherwise.

LP Formulation for the First Scenario with a Dummy Person

Objective:

$$\begin{aligned} \text{Minimize } Z = & 4X_{CS} + 6X_{CF} + 4X_{CD} + 4X_{CW} + \\ & 3X_{PS} + 4X_{PF} + 2X_{PD} + 3X_{PW} + \\ & 7X_{ES} + 5X_{EF} + 6X_{ED} + 5X_{EW} + \\ & M(X_{DS} + X_{DF} + X_{DD} + X_{DW}) \end{aligned}$$

Subject to:

$$\begin{aligned} X_{CS} + X_{CF} + X_{CD} + X_{CW} &= 1 \\ X_{PS} + X_{PF} + X_{PD} + X_{PW} &= 1 \\ X_{ES} + X_{EF} + X_{ED} + X_{EW} &= 1 \\ X_{DS} + X_{DF} + X_{DD} + X_{DW} &= 1 \\ X_{CS} + X_{PS} + X_{ES} + X_{DS} &= 1 \\ X_{CF} + X_{PF} + X_{EF} + X_{DF} &= 1 \\ X_{CD} + X_{PD} + X_{ED} + X_{DD} &= 1 \\ X_{CW} + X_{PW} + X_{EW} + X_{DW} &= 1 \\ X_{ij} &\geq 0 \quad \text{and integer (for all } i, j) \end{aligned}$$

Where X_{ij} is 1 if person i (including the dummy person D) is assigned to task j , and 0 otherwise, and M is a large number (penalty for assigning a task to the dummy).

Network Optimization Models Study Set

3/25

LP Formulation for the Second Scenario

Objective:

$$\begin{aligned} \text{Minimize } Z = & 4X_{CS} + 6X_{CF} + 4X_{CD} + 4X_{CW} + \\ & 3X_{PS} + 4X_{PF} + 2X_{PD} + 3X_{PW} + \\ & 7X_{ES} + 5X_{EF} + 6X_{ED} + 5X_{EW} + \end{aligned}$$

Subject to:

$$X_{CS} + X_{CF} + X_{CD} + X_{CW} \leq 2$$

$$X_{PS} + X_{PF} + X_{PD} + X_{PW} \leq 2$$

$$X_{ES} + X_{EF} + X_{ED} + X_{EW} \leq 2$$

$$X_{CS} + X_{PS} + X_{ES} = 1$$

$$X_{CF} + X_{PF} + X_{EF} = 1$$

$$X_{CD} + X_{PD} + X_{ED} = 1$$

$$X_{CW} + X_{PW} + X_{EW} = 1$$

$$X_{ij} \geq 0 \quad \text{and integer (for all } i, j)$$

Network Optimization Models Study Set

4/25

```
#Scenario 1
people = ['Carpenter', 'Plumber', 'Engineer']
tasks = ['SOLDERING', 'FRAMING', 'DRAFTING', 'WIRING']
inefficiency = {
    ('Carpenter', 'SOLDERING'): 4,
    ('Carpenter', 'FRAMING'): 6,
    ('Carpenter', 'DRAFTING'): 4,
    ('Carpenter', 'WIRING'): 4,
    ('Plumber', 'SOLDERING'): 3,
    ('Plumber', 'FRAMING'): 4,
    ('Plumber', 'DRAFTING'): 2,
    ('Plumber', 'WIRING'): 3,
    ('Engineer', 'SOLDERING'): 7,
    ('Engineer', 'FRAMING'): 5,
    ('Engineer', 'DRAFTING'): 6,
    ('Engineer', 'WIRING'): 5,
}

# Scenario 1
model1 = Model("Scenario_1")
x1 = model1.addVars(people, tasks, vtype=GRB.BINARY, name="x1")

# Objective
model1.setObjective(sum(inefficiency[p, t] * x1[p, t] for p in people for t
    in tasks), GRB.MINIMIZE)

# Constraints
for p in people:
    model1.addConstr(sum(x1[p, t] for t in tasks) == 1, f"Person_{p}
        _Task_Limit")
for t in tasks:
    model1.addConstr(sum(x1[p, t] for p in people) <= 1, f"Task_{t}
        _Assigned_Limit")

model1.optimize()

if model1.status == GRB.OPTIMAL:
    print("\nScenario_1_Results:")
    for p in people:
        for t in tasks:
            if x1[p, t].x > 0.5:
                print(f"{p}_assigned_to_{t}")
elif model1.status == GRB.INFEASIBLE:
    print("\nScenario_1_Model_is_infeasible.")
else:
    print(f"\nScenario_1_Optimization_ended_with_status_{model1.status}")
```

Network Optimization Models Study Set

5/25

```
# Scenario 1 with Dummy
people = ['Carpenter', 'Plumber', 'Engineer', 'Dummy']
tasks = ['SOLDERING', 'FRAMING', 'DRAFTING', 'WIRING']
inefficiency = {
    ('Carpenter', 'SOLDERING'): 4,
    ('Carpenter', 'FRAMING'): 6,
    ('Carpenter', 'DRAFTING'): 4,
    ('Carpenter', 'WIRING'): 4,
    ('Plumber', 'SOLDERING'): 3,
    ('Plumber', 'FRAMING'): 4,
    ('Plumber', 'DRAFTING'): 2,
    ('Plumber', 'WIRING'): 3,
    ('Engineer', 'SOLDERING'): 7,
    ('Engineer', 'FRAMING'): 5,
    ('Engineer', 'DRAFTING'): 6,
    ('Engineer', 'WIRING'): 5,
    ('Dummy', 'SOLDERING'): 100000000,
    ('Dummy', 'FRAMING'): 100000000,
    ('Dummy', 'DRAFTING'): 100000000,
    ('Dummy', 'WIRING'): 100000000,
}

model = Model('Q1')
X = model.addVars(people, tasks, vtype = GRB.BINARY)

model.setObjective(sum(inefficiency[p,t]*X[p,t] for p in people for t in
    tasks), GRB.MINIMIZE)

for p in people:
    model.addConstr(sum(X[p,t] for t in tasks) == 1)

for t in tasks:
    model.addConstr(sum(X[p,t] for p in people) == 1)

model.optimize()

for p in people:
    for t in tasks:
        if X[p,t].x > 0:
            print(f"{p}_assigned_to_{t}")
```

Network Optimization Models Study Set

6/25

```
# Scenario 2

people = ['Carpenter', 'Plumber', 'Engineer']
tasks = ['SOLDERING', 'FRAMING', 'DRAFTING', 'WIRING']
inefficiency = {
    ('Carpenter', 'SOLDERING'): 4,
    ('Carpenter', 'FRAMING'): 6,
    ('Carpenter', 'DRAFTING'): 4,
    ('Carpenter', 'WIRING'): 4,
    ('Plumber', 'SOLDERING'): 3,
    ('Plumber', 'FRAMING'): 4,
    ('Plumber', 'DRAFTING'): 2,
    ('Plumber', 'WIRING'): 3,
    ('Engineer', 'SOLDERING'): 7,
    ('Engineer', 'FRAMING'): 5,
    ('Engineer', 'DRAFTING'): 6,
    ('Engineer', 'WIRING'): 5,
}

model = Model('Q1_2')
X = model.addVars(people, tasks, vtype = GRB.BINARY)

model.setObjective(sum(inefficiency[p,t]*X[p,t] for p in people for t in
    tasks), GRB.MINIMIZE)

for p in people:
    model.addConstr(sum(X[p,t] for t in tasks) <= 2)

for t in tasks:
    model.addConstr(sum(X[p,t] for p in people) == 1)

model.optimize()

for p in people:
    for t in tasks:
        if X[p,t].x > 0:
            print(f"{p}_assigned_to_{t}")
```

Network Optimization Models Study Set

7/25

Question 2. (Modified from Hamdy A. Taha (2017) ^a)

E-machine is developing a replacement policy for its equipment fleet over a 4-year planning horizon. At the start of each year, a equipment is either replaced or kept in operation for an extra year. An equipment must be in service from 1 to 3 years. The following table 2 provides the replacement cost as a function of the year a machine is acquired and the number of years in operation. What is the optimal replacement strategy for E-machine?

^aHamdy A. Taha (2017). Operations Research An Introduction)

Equipment acquired at start of year	Replacement cost (\$) for given years in operation		
	1	2	3
1	4000	5400	9800
2	4300	6200	8700
3	4800	7100	—
4	4900	—	—

Table 2: The replacement cost as a function of the year an equipment is acquired and the number of years in operation

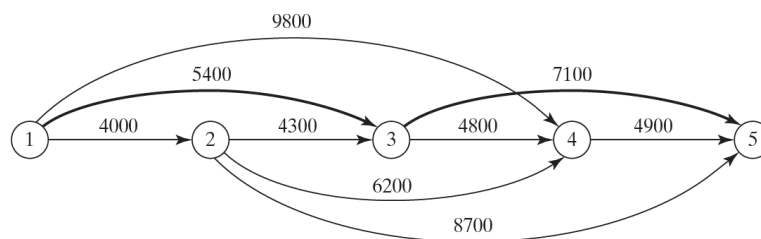


Figure 1: Network Representation of E-machine Replacement Process

Network Optimization Models Study Set

8/25

Correct Answer 2.

The problem can be formulated as a network in which nodes 1 to 5 represent the start of years 1 to 5. Arcs from node 1 (year 1) can reach nodes 2, 3, and 4 because an equipment must be in operation from 1 to 3 years. The arcs from the other nodes can be interpreted similarly. The length of each arc equals the replacement cost. The solution of the problem is equivalent to finding the shortest route between nodes 1 and 5.

Decision Variables

For each node pair (i, j) :

$$x_{i,j} = \begin{cases} 1 & \text{if the path from node } N_i \text{ to node } N_j \text{ is taken} \\ 0 & \text{otherwise} \end{cases}$$

Objective Function

Minimize the total cost over the 4-year planning horizon:

$$\begin{aligned} \text{Minimize } Z = & 4000x_{1,2} + 4300x_{2,3} + 4800x_{3,4} + 4900x_{4,5} \\ & + 5400x_{1,3} + 6200x_{2,4} + 7100x_{3,5} \\ & + 9800x_{1,4} + 8700x_{2,5} \end{aligned}$$

Constraints

Ensure that exactly one path is taken leaving each node (except the final node):

$$\begin{aligned} x_{1,2} + x_{1,3} + x_{1,4} &= 1 \\ x_{2,3} + x_{2,4} + x_{2,5} &= 1 \\ x_{1,2} - (x_{2,3} + x_{2,4} + x_{2,5}) &= 0 \\ x_{2,3} + x_{1,3} - (x_{3,4} + x_{3,5}) &= 0 \\ x_{3,4} + x_{1,4} + x_{2,4} - x_{4,5} &= 0 \end{aligned}$$

The path continuity constraint:

$$x_{1,5} = 0 \quad (\text{No direct path from } N_1 \text{ to } N_5)$$

Network Optimization Models Study Set

9/25

```
# Create a new Gurobi model
model = Model("machine_replacement")

# Decision variables
nodes = 5 # Number of nodes (N1, N2, N3, N4, N5)
x = {}
for i in range(1, nodes + 1):
    for j in range(1, nodes + 1):
        x[i, j] = model.addVar(vtype=GRB.BINARY, name=f'x_{i}_{j}')

# Define the objective function
cost = {
    (1, 2): 4000,
    (2, 3): 4300,
    (3, 4): 4800,
    (4, 5): 4900,
    (1, 3): 5400,
    (2, 4): 6200,
    (3, 5): 7100,
    (1, 4): 9800,
    (2, 5): 8700,
}

obj = sum(cost[i, j] * x[i, j] for i, j in cost)
model.setObjective(obj, GRB.MINIMIZE)

# Constraints
# Ensure one path is taken leaving each node (except the final node)
model.addConstr(x[1, 2] + x[1, 3] + x[1, 4] == 1, name='node_out_1')
model.addConstr(x[2, 5] + x[3, 5] + x[4, 5] == 1, name='node_out_5')
model.addConstr(x[1, 2] - (x[2, 3] + x[2, 4] + x[2, 5]) == 0, name='
    node_out_2')
model.addConstr(x[2, 3] + x[1, 3] - (x[3, 4] + x[3, 5]) == 0, name='
    node_out_3')
model.addConstr(x[3, 4] + x[1, 4] + x[2, 4] - x[4, 5] == 0, name='
    node_out_4')

# Path continuity constraint
model.addConstr(x[1, 5] == 0, name='no_direct_path_1_4_5')

# Optimize the model
model.optimize()

# Display the solution
if model.status == GRB.OPTIMAL:
    print("Optimal solution found")
    for i in range(1, nodes + 1):
        for j in range(1, nodes + 1):
            if i != j and x[i, j].x > 0.5:
                print(f"Take path from N_{i} to N_{j}")
    print(f"Total Cost: {model.objVal}")
else:
    print("No optimal solution found")
```

Network Optimization Models Study Set

10/25

Question 3. *The academic council at the Columbia University is seeking representation from among six students who are affiliated with four honor societies. The academic council representation includes three areas: mathematics, art, and engineering. At most two students in each area can be on the council. The table 3 shows the membership of the six students in the four honor societies. The students who are skilled in the areas of mathematics, art, and engineering are shown in table 4. A student who is skilled in more than one area must be assigned exclusively to one area only. Can all four honor societies be represented on the council?*

Society	Affiliated students
1	1, 2, 3, 4
2	1, 3, 6
3	2, 3, 4, 5
4	1, 2, 4, 6

Table 3: The membership of the six students in the four honor societies

Area Skilled	Students
Mathematics	1, 2, 3, 4
Art	1, 3, 4, 5
Engineering	1, 4, 5, 6

Table 4: The students who are skilled in the areas of mathematics, art, and engineering

Network Optimization Models Study Set

11/25

Correct Answer 3.

Decision Variables: Let x_{ij} be a binary variable that equals 1 if student i represents area j and 0 otherwise. Here, i varies from 1 to 6 (representing each student) and j varies from 1 to 3 (representing each area: mathematics, art, engineering).

Objective Function: Our goal is to determine if all honor societies can be represented. For simplicity, let the objective function be the sum of all x_{ij} .

$$\max \sum_{i=1}^6 \sum_{j=1}^3 x_{ij}$$

Constraints:

1. Each student can represent at most one area:

$$\sum_{j=1}^3 x_{ij} \leq 1 \quad \forall i$$

2. At most two students in each area:

$$\sum_{i=1}^6 x_{ij} \leq 2 \quad \forall j$$

3. Students' skills constraints.

For example, only students 1, 2, 3, and 4 can represent Mathematics.

$$x_{51} = x_{61} = 0$$

Hence, for Art:

$$x_{22} = x_{62} = 0$$

And for Engineering:

$$x_{23} = x_{3,3} = 0$$

4. To ensure every society is represented. For instance, for society 1:

$$x_{11} + x_{12} + x_{13} + x_{21} + x_{22} + x_{23} + x_{31} + x_{32} + x_{33} + x_{41} + x_{42} + x_{43} \geq 1$$

Similar constraints should be formulated for societies 2, 3, and 4.

Finally, the decision variables are binary:

$$x_{ij} \in \{0, 1\} \quad \forall i, j$$

Network Optimization Models Study Set

12/25

```
# Create a new model
m = Model("honor_societies_representation")

# Decision variables
x = {}
for i in range(1, 7): # Students
    for j in range(1, 4): # Areas (Mathematics, Art, Engineering)
        x[i, j] = m.addVar(vtype=GRB.BINARY, name=f"x_{i}_{j}")

# Objective function: Maximize the sum of all x_ij (though our main goal is
# to find a feasible solution)
m.setObjective(sum(x[i, j] for i in range(1, 7) for j in range(1, 4)), GRB.
    MAXIMIZE)

# Constraints
# 1. Each student can represent at most one area
for i in range(1, 7):
    m.addConstr(sum(x[i, j] for j in range(1, 4)) <= 1)

# 2. At most two students in each area
for j in range(1, 4):
    m.addConstr(sum(x[i, j] for i in range(1, 7)) <= 2)

# 3. Students' skills constraints
# Math
m.addConstr(x[5, 1] == 0)
m.addConstr(x[6, 1] == 0)
# Art
m.addConstr(x[2, 2] == 0)
m.addConstr(x[6, 2] == 0)
# Engineering
m.addConstr(x[2, 3] == 0)
m.addConstr(x[3, 3] == 0)

# 4. Each society should be represented
# Society 1
m.addConstr(sum(x[i, j] for i in [1, 2, 3, 4] for j in range(1, 4)) >= 1)
# Society 2
m.addConstr(sum(x[i, j] for i in [1, 3, 6] for j in range(1, 4)) >= 1)
# Society 3
m.addConstr(sum(x[i, j] for i in [2, 3, 4, 5] for j in range(1, 4)) >= 1)
# Society 4
m.addConstr(sum(x[i, j] for i in [1, 2, 4, 6] for j in range(1, 4)) >= 1)

# Solve
m.optimize()

if m.status == GRB.OPTIMAL:
    print("Solution found!")
    for i in range(1, 7):
        for j in range(1, 4):
            if x[i, j].x == 1:
                print(f"Student_{i}_represents_area_{j}")
else:
    print("No solution found")
```

Network Optimization Models Study Set

13/25

Question 4. *Chicken feed is transported by trucks from three silos to four farms. Some of the silos cannot ship directly to some of the farms. The capacities of the other routes are limited by the number of trucks available and the number of trips made daily. The following table 5 shows the daily amounts of supply at the silos and demand at the farms (in thousands of pounds). The cell entries of the table specify the daily capacities of the associated routes.*

Determine the schedule that satisfies the most demand.

Will the proposed schedule satisfy all the demand at the farms?

		Farm				
		1	2	3	4	
Silo	1	30	5	0	40	20
	2	0	0	5	90	20
	3	100	40	30	40	200
		200	10	60	20	

Table 5: The daily amounts of supply at the silos and demand at the farms

Network Optimization Models Study Set

14/25

Correct Answer 4. Given the problem of transporting chicken feed from silos to farms, we can model this as a linear programming problem. Let x_{ij} represent the amount of feed transported from silo i to farm j .

Objective Function:

$$\text{Maximize } Z = \sum_{i=1}^3 \sum_{j=1}^4 x_{ij} \quad (1)$$

Constraints:

1. Supply constraints for each silo:

$$\sum_{j=1}^4 x_{1j} \leq 20,000 \quad (2)$$

$$\sum_{j=1}^4 x_{2j} \leq 20,000 \quad (3)$$

$$\sum_{j=1}^4 x_{3j} \leq 200,000 \quad (4)$$

2. Demand constraints for each farm:

$$\sum_{i=1}^3 x_{i1} \leq 200,000 \quad (5)$$

$$\sum_{i=1}^3 x_{i2} \leq 10,000 \quad (6)$$

$$\sum_{i=1}^3 x_{i3} \leq 60,000 \quad (7)$$

$$\sum_{i=1}^3 x_{i4} \leq 20,000 \quad (8)$$

3. Capacity constraints for each route:

$$x_{11} \leq 30,000, \quad x_{12} \leq 5,000, \quad x_{13} \leq 0, \quad x_{14} \leq 40,000 \quad (9)$$

$$x_{21} \leq 0, \quad x_{22} \leq 0, \quad x_{23} \leq 5,000, \quad x_{24} \leq 90,000 \quad (10)$$

$$x_{31} \leq 100,000, \quad x_{32} \leq 40,000, \quad x_{33} \leq 30,000, \quad x_{34} \leq 40,000 \quad (11)$$

4. Non-negativity constraints:

$$x_{ij} \geq 0, \quad \forall i, j \quad (12)$$

By solving this LP model, we can determine the transportation schedule that satisfies the most demand at the farms.

Network Optimization Models Study Set

15/25

```
# Create a new model
m = Model("chicken_feed_transportation")

# Indices for silos and farms
silos = [1, 2, 3]
farms = [1, 2, 3, 4]

# Supply at each silo
supply = {1: 20000, 2: 20000, 3: 200000}

# Demand at each farm
demand = {1: 200000, 2: 10000, 3: 60000, 4: 20000}

# Transportation capacities from silo i to farm j
capacity = {(1, 1): 30000, (1, 2): 5000, (1, 3): 0, (1, 4): 40000,
            (2, 1): 0, (2, 2): 0, (2, 3): 5000, (2, 4): 90000,
            (3, 1): 100000, (3, 2): 40000, (3, 3): 30000, (3, 4): 40000}

# Variables for amount of feed transported from silo i to farm j
x = m.addVars(silos, farms, name="x", ub=capacity)

# Set objective to maximize demand satisfied
m.setObjective(x.sum('*', '*'), GRB.MAXIMIZE)

# Add supply constraints for each silo
m.addConstrs((x.sum(i, '*') <= supply[i] for i in silos), name="supply")

# Add demand constraints for each farm
m.addConstrs((x.sum('*', j) <= demand[j] for j in farms), name="demand")

# Solve the model
m.optimize()

# Print the solution
if m.status == GRB.OPTIMAL:
    print("\nOptimal solution found!")
    for i in silos:
        for j in farms:
            if x[i, j].x > 1e-6: # Printing non-zero values
                print(f"Transport {x[i, j].x} from Silo {i} to Farm {j}")
else:
    print("\nNo solution found!")

# Check if all demand is satisfied
total_demand_satisfied = sum(x.sum('*', j).getValue() for j in farms)
if total_demand_satisfied >= sum(demand.values()):
    print("\nThe proposed schedule satisfies all the demand at the farms!")
else:
    print(f"\nThe proposed schedule satisfies {total_demand_satisfied} out of {sum(demand.values())} total demand.")
```

Network Optimization Models Study Set

16/25

Question 5. Three refineries send a gasoline product to two distribution terminals through a pipeline network. Any demand that cannot be satisfied through the network is acquired from other sources. The pipeline network is served by three pumping stations, as shown in Figure 2. The product flows in the network in the direction shown by the arrows. The capacity of each pipe segment (shown directly on the arcs) is in million bbl per day. Determine the following:

- The daily production at each refinery that matches the maximum capacity of the network.
- The daily demand at each terminal that matches the maximum capacity of the network.
- The daily capacity of each pump that matches the maximum capacity of the network.

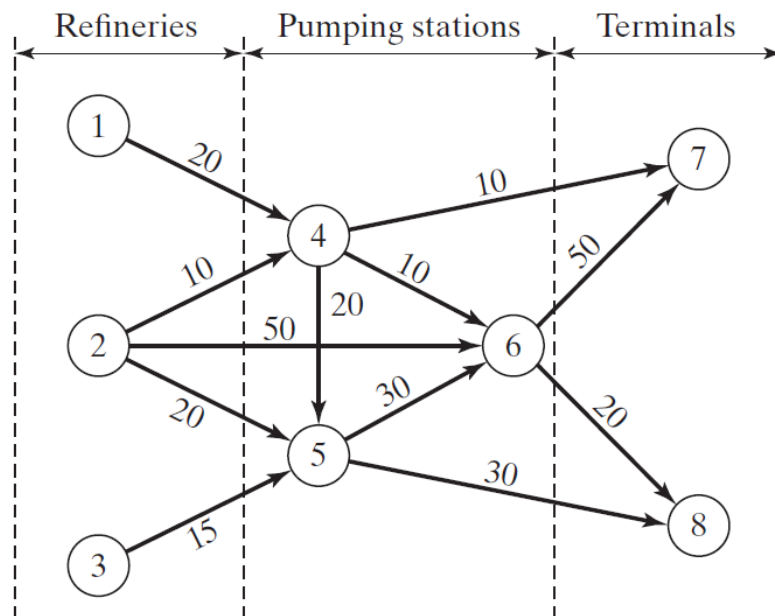
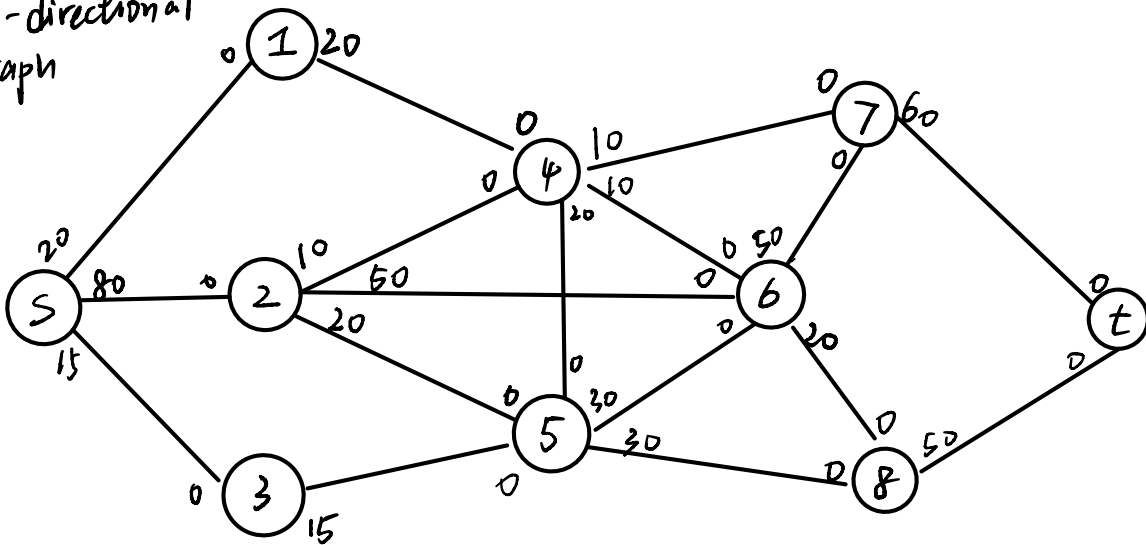


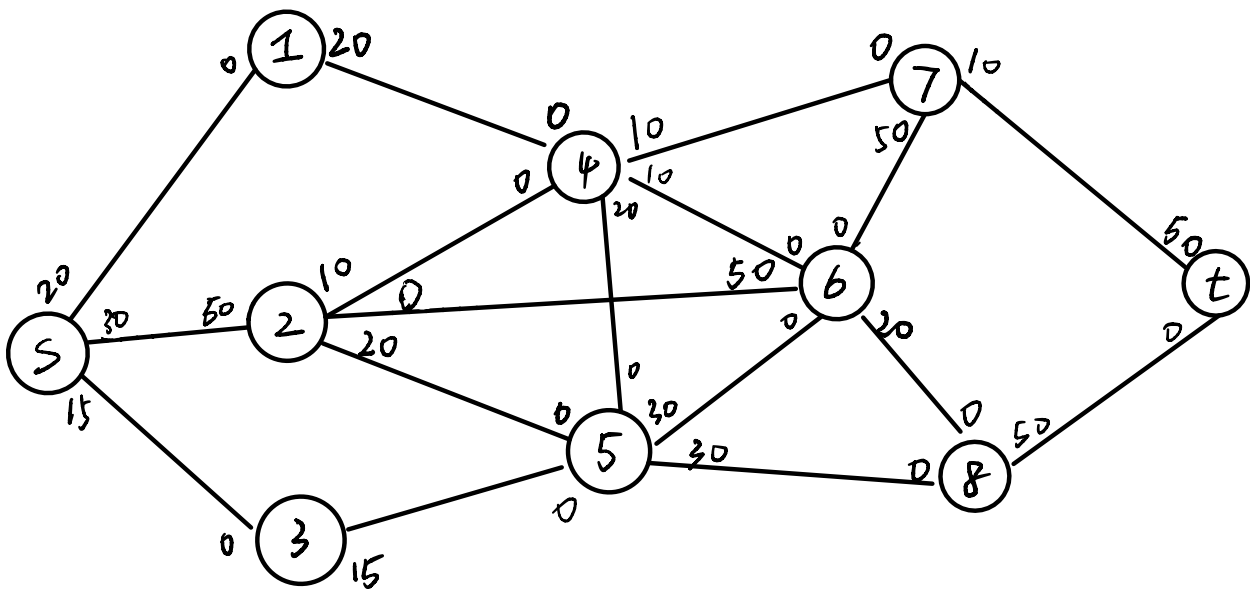
Figure 2: The pipeline network

Correct Answer 5. Note that there are multiple optimal solutions. To model the question as a standard max-flow problem, we need to add two dummy nodes s and t that represent production and demand, respectively. Then, it suffices to solve the $s-t$ max-flow problem. Here's an example using the Ford-Fulkerson Algorithm.

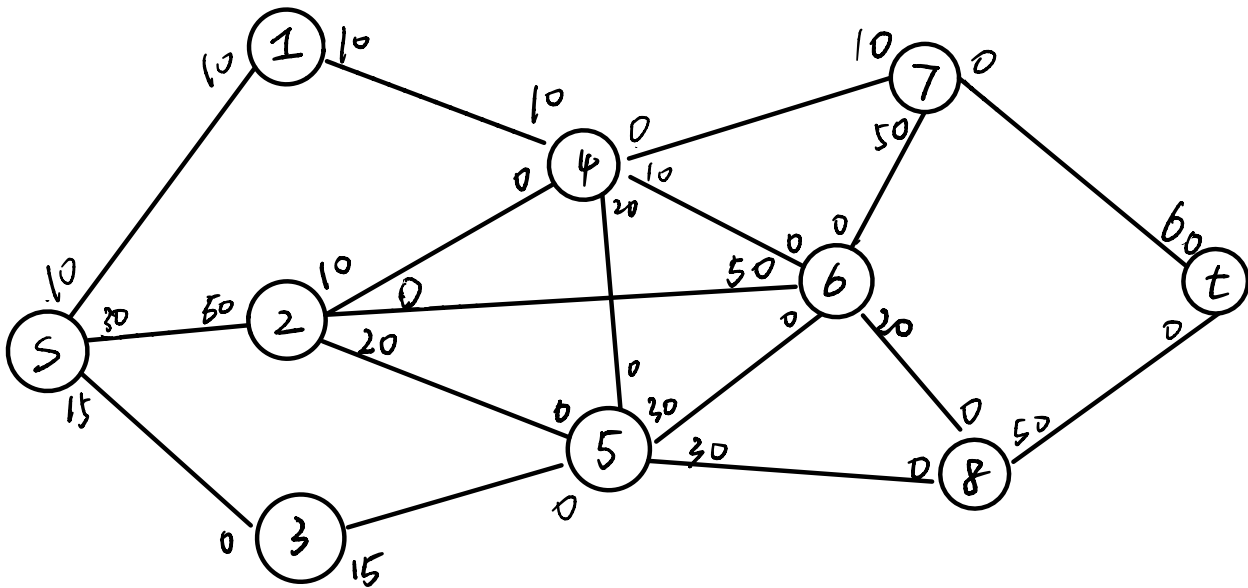
1. Bi-directional graph



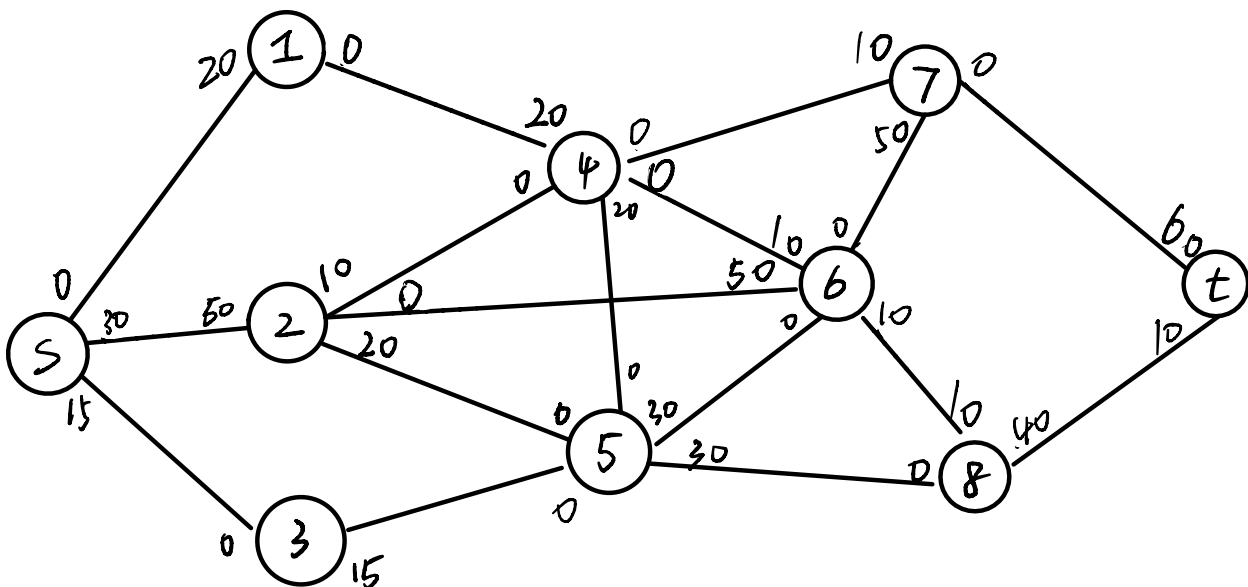
2. $S \rightarrow 2 \rightarrow 6 \rightarrow 7 \rightarrow t$ residual capacity = 50
 current flow value $F = 50$



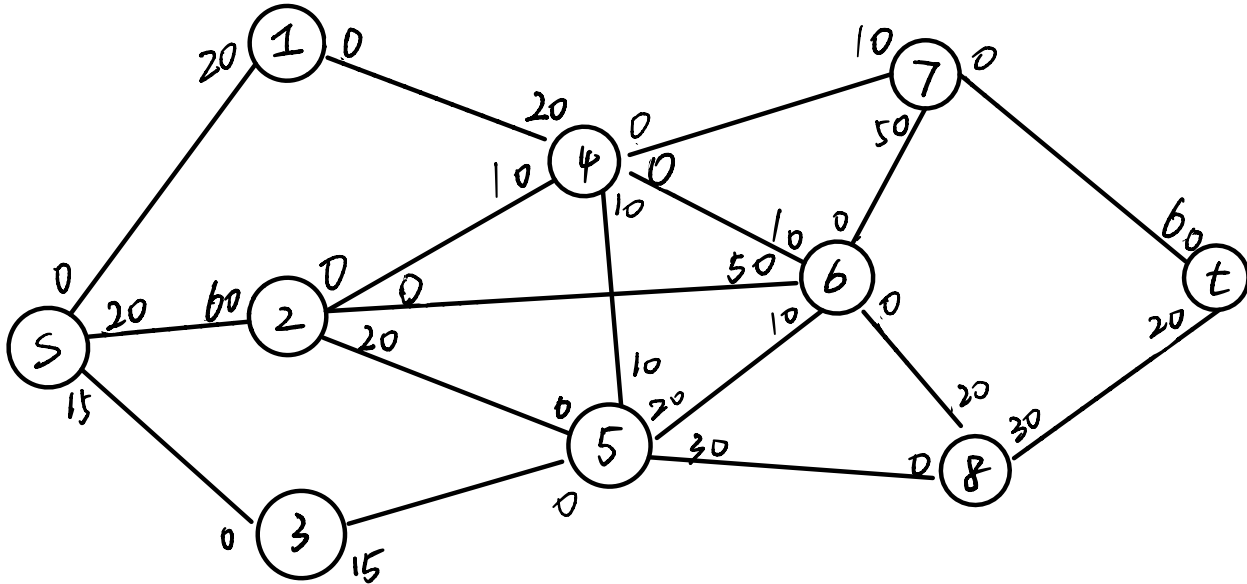
3. $s \rightarrow 1 \rightarrow 4 \rightarrow 7 \rightarrow t$ residual capacity = 10, current flow value $Z = 60$



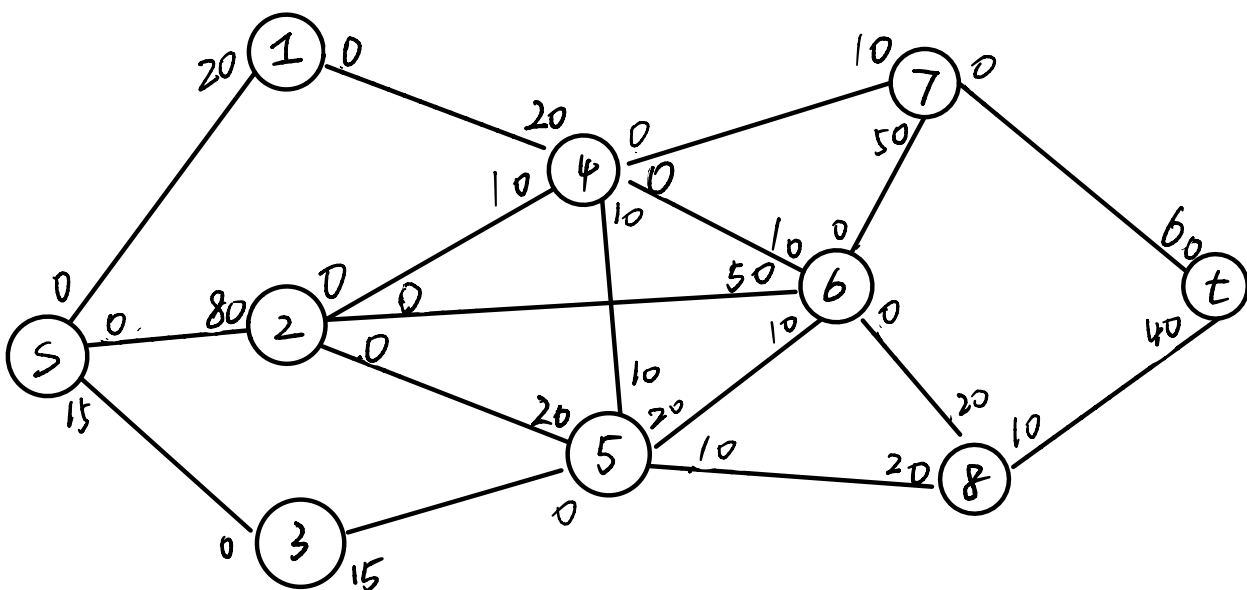
4. $s \rightarrow 1 \rightarrow 4 \rightarrow 6 \rightarrow 8 \rightarrow t$ residual capacity = 10
current flow value $Z = 70$



5. $s \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 8 \rightarrow t$ residual capacity = 10
current flow value $Z = 80$



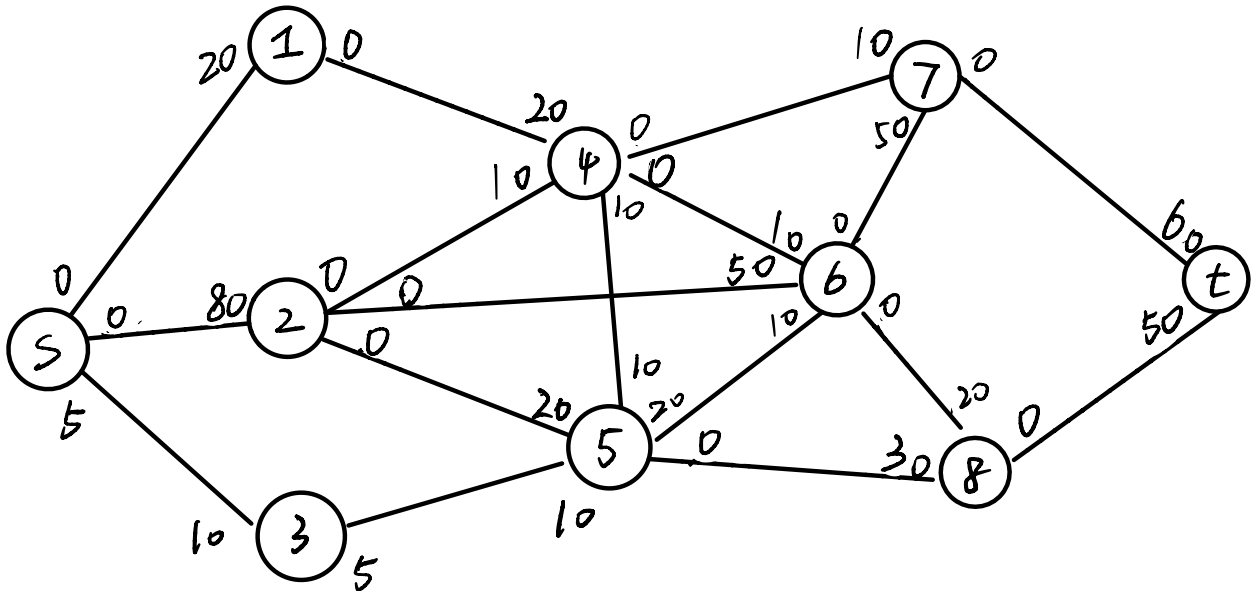
6. $s \rightarrow 2 \rightarrow 5 \rightarrow 8 \rightarrow t$ residual capacity = 20
current flow value $Z = 100$



7 $s \rightarrow 3 \rightarrow 5 \rightarrow 8 \rightarrow t$

residual capacity = 10

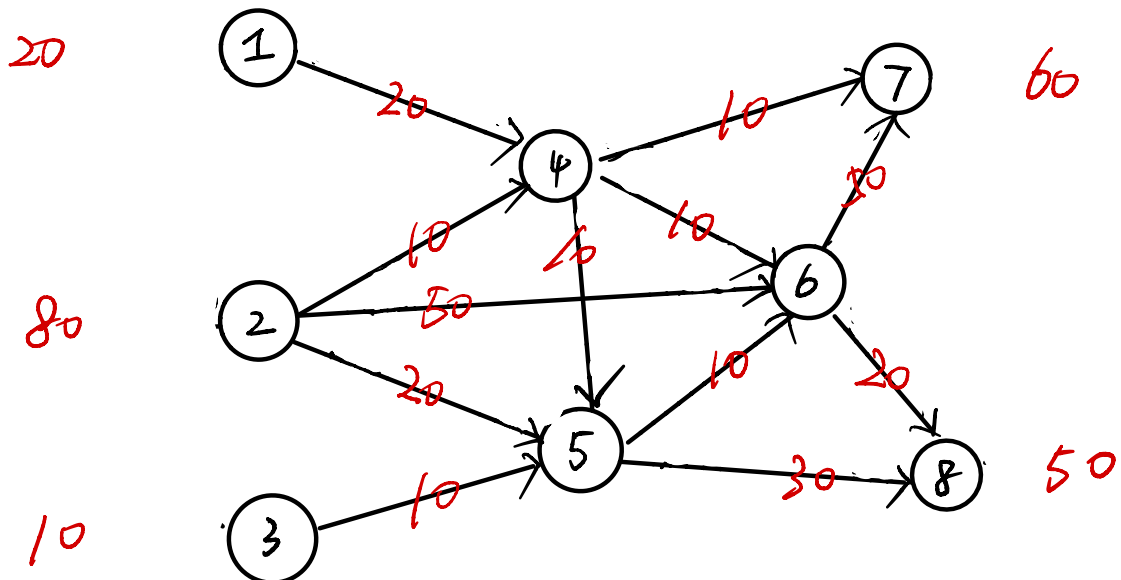
current flow value $z = 110$



8 final solution

production

demand



Network Optimization Models Study Set

21/25

```
# Create the model
m = Model("max_flow")

# Add decision variables
x = {}
arcs = [(1,4), (2,4), (2,6), (2,5), (3,5), (4,5), (4,6), (4,7), (5,6),
        (5,8), (6,7), (6,8)]
capacities = { (1,4): 20, (2,4):10, (2,6):50, (2,5):20, (3,5):15, (4,5):20,
               (4,6):10, (4,7):10,
               (5,6):30, (5,8):30, (6,7):50, (6,8):20 }

for arc in arcs:
    x[arc] = m.addVar(vtype=GRB.CONTINUOUS, name=f"x_{arc[0]}{arc[1]}")

# Set the objective function
m.setObjective(x[4,7] + x[5,8] + x[6,7] + x[6,8], GRB.MAXIMIZE)

# Add constraints
m.addConstr(x[1,4]+ x[2,4] == x[4,5] + x[4,6] + x[4,7])
m.addConstr(x[2,5] + x[3,5] + x[4,5] == x[5,6] + x[5,8])
m.addConstr(x[2,6] + x[4,6] + x[5,6] == x[6,7] + x[6,8])

for arc, capacity in capacities.items():
    m.addConstr(x[arc] <= capacity)

# Solve the model
m.optimize()

# Print the solution
if m.status == GRB.Status.OPTIMAL:
    for arc in arcs:
        print(f"Flow on arc {arc}: {x[arc].x}")
else:
    print("No solution found!")
```

Network Optimization Models Study Set

22/25

Question 6. Determine the maximal flow and the optimum flow in each arc for the network given below in Figure 3.

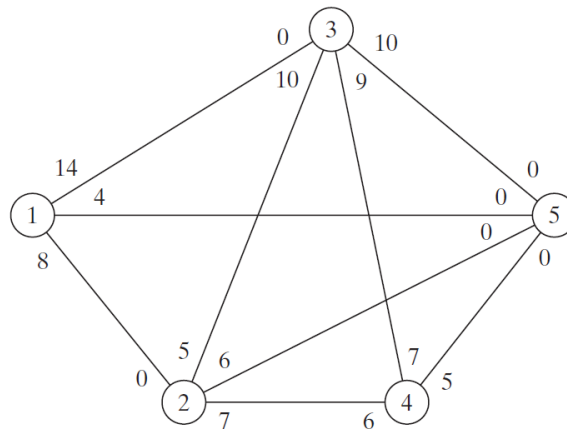
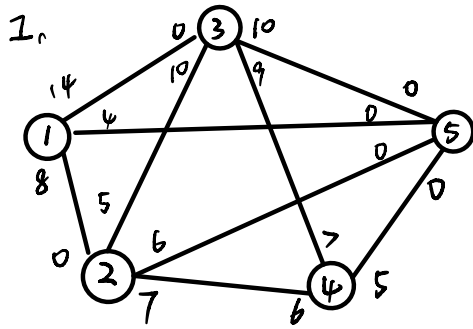
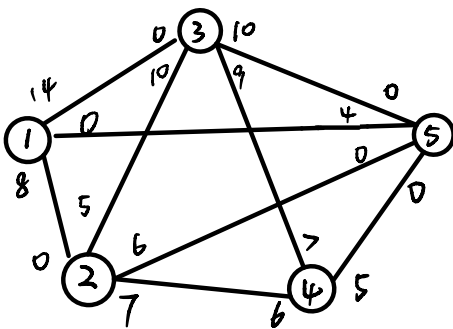


Figure 3: Network for Question 6

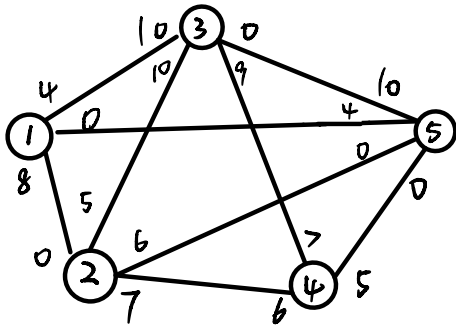
CorrectAnswer 6. We apply the Ford-Fulkerson Algorithm to find the max-flow from 1 to 5.



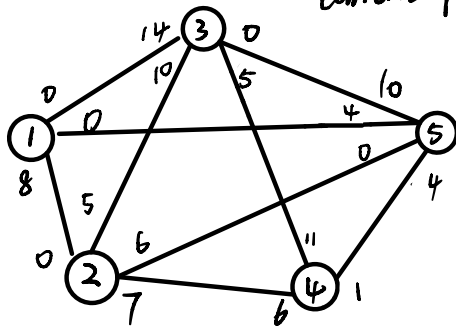
2. $1 \rightarrow 5$ residual capacity 4
current flow value $z = 4$



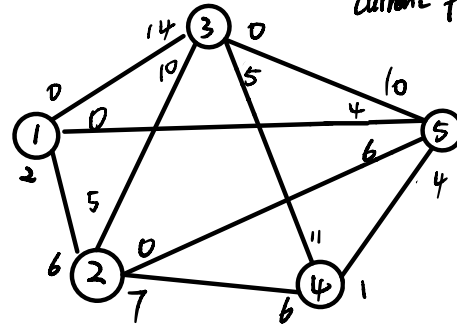
3. $1 \rightarrow 3 \rightarrow 5$ residual capacity = 10
current flow value $z = 14$



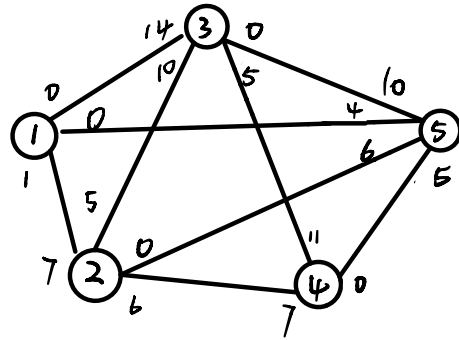
4. $1 \rightarrow 3 \rightarrow 4 \rightarrow 5$ residual capacity = 4
current flow value $z = 18$



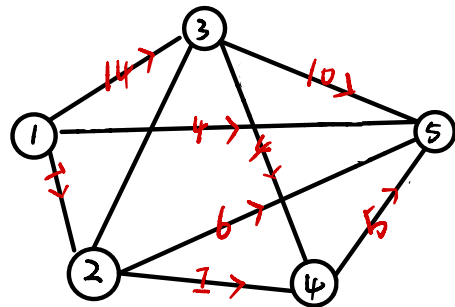
5. $1 \rightarrow 2 \rightarrow 5$ residual capacity = 6
current flow value $z = 24$



6. $1 \rightarrow 2 \rightarrow 4 \rightarrow 5$ residual capacity = 1
current flow value $z = 25$



7. final solution



Network Optimization Models Study Set

24/25

Question 7. The Tell-All mobile-phone company services six geographical areas. The satellite distances (in miles) among the six areas are given in Figure 4. Tell-All needs to determine the most efficient message routes that should be established between each two areas in the network.

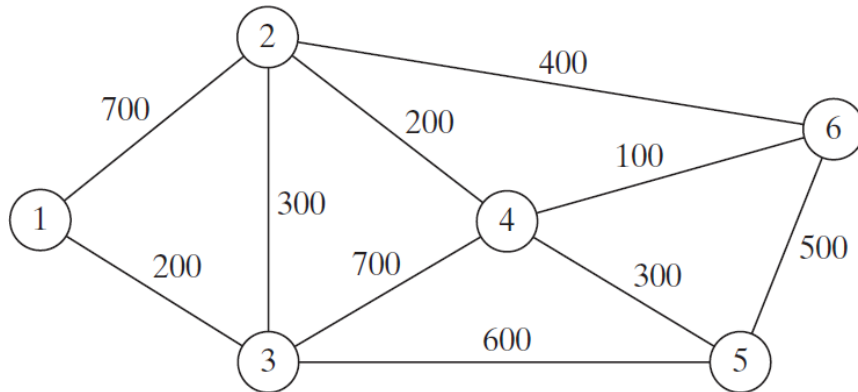


Figure 4: The satellite distances (in miles) among the six areas

Correct Answer 7. Finding the most efficient message routes is equivalent to solving the minimum spanning tree (MST) problem over the graph in Figure 4.

We apply the Kruskal algorithm: Starting from an empty tree $\{\}$ we select edges in increasing order of length and skip edges that would create a cycle with the edges already selected. We get $\{\} \rightarrow \{46\} \rightarrow \{46, 24\} \rightarrow \{46, 24, 13\} \rightarrow \{46, 24, 13, 45\} \rightarrow \{46, 24, 13, 45, 23\}$. Hence the set of edges $\{46, 24, 13, 45, 23\}$ is a spanning tree and are (an example of) the most efficient set of message routes that Tell-All can use.

Decision Variables:

For each arc (i,j) , let:

$$x_{ij} = \begin{cases} 1 & \text{if arc } (i,j) \text{ is in the MST} \\ 0 & \text{otherwise} \end{cases}$$

For each node i , representing its position in the MST:

$$y_i \in \{1, 2, \dots, n\}$$

$$\begin{aligned} \min \quad & \sum_{(i,j) \in \text{arcs}} c_{ij} x_{ij} \\ \text{s.t.} \quad & \sum_{(i,j) \in \text{arcs}} x_{ij} = n - 1 \\ & y_i - y_j + 1 \leq (n - 1)(1 - x_{ij}) \quad \forall (i,j) \in \text{arcs} \\ & y_j - y_i + 1 \leq (n - 1)(1 - x_{ij}) \quad \forall (i,j) \in \text{arcs} \\ & y_i \in \{1, 2, \dots, n\} \quad \forall i \in \text{nodes} \\ & x_{ij} \in \{0, 1\} \quad \forall (i,j) \in \text{arcs} \end{aligned}$$

Network Optimization Models Study Set

25/25

```
#Kruskal's Algorithm
# Data: edges with weights
edges = {
    (1, 2): 700,
    (1, 3): 200,
    (2, 3): 300,
    (2, 4): 200,
    (3, 4): 700,
    (2, 6): 400,
    (3, 5): 600,
    (4, 5): 300,
    (4, 6): 100,
    (5, 6): 500
}

# Sorting edges based on weight
sorted_edges = sorted(edges.items(), key=lambda item: item[1])

# Data structure to keep track of the set each element belongs to
parent = {i: i for i in range(1, 7)} # Initially, each node is its own
rank = {i: 0 for i in range(1, 7)} # Used to keep the tree flat

def find(node):
    """Find the set of the node with path compression."""
    if parent[node] != node:
        parent[node] = find(parent[node])
    return parent[node]

def union(node1, node2):
    """Union the sets of the two nodes."""
    root1 = find(node1)
    root2 = find(node2)

    # Attach the smaller rank tree under the root of the higher rank tree
    if root1 != root2:
        if rank[root1] > rank[root2]:
            parent[root2] = root1
        else:
            parent[root1] = root2
            if rank[root1] == rank[root2]:
                rank[root2] += 1

mst = [] # To store the edges of the MST

# Kruskal's Algorithm
for edge, weight in sorted_edges:
    if find(edge[0]) != find(edge[1]):
        union(edge[0], edge[1])
        mst.append(edge)

print("Edges in the Minimum Spanning Tree:")
for edge in mst:
    print(edge)
```